JavaScript Fundamentals

Core language concepts, modern syntax, functions, objects and modules

Learning Outcomes

- Explain what JavaScript does in a web page.
- Use var, let and const correctly.
- Work with primitive and reference values.
- Use operators, comparisons, conditions, loops and switch.
- Explain scope, hoisting and the temporal dead zone.
- Write function declarations, expressions and arrow functions.
- Use default/rest parameters, spread and destructuring.
- Understand this, closures, objects, arrays, template literals and ES6 modules.

1. Role of JavaScript & Embedding Scripts

- HTML gives the page structure; CSS controls appearance; JavaScript adds behavior.
- JavaScript can react to clicks, validate forms, change content and communicate with servers.
- Inline script: written inside an HTML element (use sparingly).
- Internal script: written inside <script> in the HTML file.
- External script: JavaScript is stored in a separate .js file.

HTML examples

```
<script>
  const btn =
document.querySelector('#btn');
  btn.textContent = 'Ready!';
</script>

<!-- External file -->
<script src="app.js"></script>
```

Good practice: keep larger JavaScript in external .js files.

Script Loading: defer and async

defer

- Downloads the script while HTML is being parsed.
- Runs after HTML parsing is complete.
- Deferred scripts keep their order.
- Useful for scripts that need the page DOM.

async

- Downloads the script while HTML is being parsed.
- Runs as soon as the download finishes.
- Order is not guaranteed.
- Useful for independent scripts such as analytics.

```
<script src="app.js" defer></script>
<script src="analytics.js" async></script>
```

2. var, let and const

- let: use when the variable value will change.
- const: use when the variable should not be reassigned.
- var: older syntax; it is function-scoped and can be redeclared.
- Modern code normally prefers const, then let when reassignment is needed.
- const does not make an object completely immutable; its reference cannot be reassigned.

Simple example

```
let score = 10;
score = 20;

const name = 'Ali';
// name = 'Sara'; // Error

var oldStyle = true;
```

Rule of thumb: start with const. Change to let only when you need reassignment.

Primitive vs. Reference Data Types

Primitive values

- string, number, bigint, boolean
- undefined, null, symbol
- Copied as values.
- Example: changing one number does not change another number.

Reference values

- Objects, arrays and functions.
- Variables hold a reference to the object.
- Two variables can refer to the same object.
- Changing the object can be visible through both variables.

Compare the behavior

```
let a = 10;
let b = a;
b = 20;
console.log(a); // 10

const user1 = { name: 'Ali' };
const user2 = user1;
user2.name = 'Sara';
console.log(user1.name); // Sara
```

3. Operators, Type Coercion and == vs. ===

- Arithmetic: +, -, *, /, %, **
- Comparison: >, <, >=, <=, ==, ===, !=, !==
- Logical: && (and), || (or), ! (not)
- Assignment: =, +=, -=, *=, /=
- Type coercion means JavaScript converts a value to another type in some operations.
- == allows type conversion; === checks value and type.

Examples

```
console.log(5 + '2');   // '52'
console.log(5 - '2');   // 3

console.log(5 == '5');  // true
console.log(5 === '5'); // false

// Prefer === in most code
if (age === 18) {
  console.log('Adult');
}
```

Use === and !== by default. They make comparisons more predictable.

Control Flow: if, else, loops and switch

Conditionals

- if runs code when a condition is true.
- else handles the other case.
- else if checks another condition.
- Use conditions to make decisions.

Loops & switch

- for: repeat a known number of times.
- while: repeat while a condition is true.
- switch: choose among several cases.
- break exits a loop or switch.

Simple control flow

```
const mark = 72;

if (mark >= 50) {
  console.log('Pass');
} else {
  console.log('Fail');
}
```

switch

```
const day = 2;
switch (day) {
  case 1: console.log('Mon');
  break;
  case 2: console.log('Tue');
  break;
```

4. Scope: Global, Function and Block

- Scope tells us where a variable can be accessed.
- Global scope: available throughout the script (but avoid unnecessary globals).
- Function scope: var declared inside a function is available inside that function.
- Block scope: let and const inside { } are available only in that block.
- Block scope helps prevent accidental variable changes.

Scope example

```
const globalName = 'Ali';

function test() {
  var x = 10;        // function scope
  if (x > 5) {
    let y = 20;      // block scope
    const z = 30;    // block scope
  }
  console.log(x);    // OK
  // console.log(y); // Error
}
```

Hoisting and the Temporal Dead Zone (TDZ)

Hoisting

- JavaScript processes declarations before executing code.
- Function declarations can be called before their definition.
- var is hoisted and starts with the value undefined.
- Hoisting does not mean every variable is safely usable before declaration.

Temporal Dead Zone

- let and const are hoisted but not initialized.
- The period before their declaration is the TDZ.
- Accessing them in the TDZ causes a ReferenceError.
- Declare variables before using them: it is clearer.

Compare

```
sayHello();

function sayHello() {
  console.log('Hello');
}

console.log(a); // undefined
```

5. Function Declarations, Expressions and Arrow Functions

- Function declaration: named function written with function.
- Function expression: function stored in a variable.
- Arrow function: shorter syntax, especially useful for callbacks.
- Functions can receive parameters and return values.
- Arrow functions do not create their own this binding.

Three common forms

```
// Declaration
function add(a, b) {
  return a + b;
}

// Expression
const multiply = function(a, b) {
  return a * b;
};

// Arrow
const square = n => n * n;

console.log(add(2, 3)); // 5
```

6. Default and Rest Parameters

Default parameter

- Provides a value when an argument is missing.
- Useful for optional settings.
- Write the default directly in the parameter list.

Rest parameter

- Collects remaining arguments into an array.
- Written with ... before the parameter name.
- Must be the last parameter.

Example

```
function greet(name = 'Guest') {
  return `Hello, ${name}`;
}

console.log(greet());
// Hello, Guest
```

Spread Operator and Destructuring

- Spread (...) expands an array/object into individual values or properties.
- Destructuring extracts values into variables.
- Array destructuring follows position.
- Object destructuring follows property names.
- These features make modern JavaScript shorter and easier to read.

Simple examples

```
const nums = [1, 2, 3];
const more = [...nums, 4, 5];

const user = { name: 'Ali', age: 20 };
const { name, age } = user;

const [first, second] = nums;

console.log(name, age);
console.log(more); // [1,2,3,4,5]
```

7. this Binding: call, apply and bind

- this refers to a value determined by how a function is called.
- obj.method(): this usually refers to obj.
- call() calls a function with a chosen this value and separate arguments.
- apply() is similar, but receives arguments as an array.
- bind() creates a new function with this fixed.

Example

```
const person = { name: 'Ali' };

function greet(city) {
  console.log(this.name + ' from ' +
city);
}

greet.call(person, 'Abbottabad');
greet.apply(person, ['Lahore']);

const fixed = greet.bind(person);
fixed('Islamabad');
```

Remember: call/apply run now; bind returns a new function to run later.

Closures: Lexical Environment, Uses and Pitfalls

- A closure happens when an inner function remembers variables from its outer scope.
- Lexical scope means the function uses the scope where it was created.
- Uses: private state, counters, callbacks and function factories.
- Pitfall: closures can keep data alive longer than expected.
- Keep closures simple and avoid unnecessary retained data.

Closure example

```
function createCounter() {
  let count = 0;

  return function () {
    count++;
    return count;
  };
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
```

The returned function still remembers count even after createCounter() has finished.

8. Objects and Arrays

Objects

- Store related data using key-value pairs.
- Access properties with dot or bracket notation.
- Methods are functions stored as object properties.
- Objects are reference values.

Arrays

- Store ordered collections.
- Indexes start at 0.
- Common methods: push, pop, map, filter, find.
- Arrays are also objects in JavaScript.

Practical example

```
const student = {
  name: 'Ayesha',
  marks: 85,
  passed: true
};

const marks = [70, 85, 90];
```

Template Literals

- Template literals use backticks: `...`
- Insert variables or expressions with ${...}.
- They make string building easier to read.
- They can contain line breaks.
- Useful for messages, HTML strings and formatted output.

Example

```
const name = 'Ali';
const marks = 85;

const message = `Student: ${name}
Marks: ${marks}
Status: ${marks >= 50 ? 'Pass' : 'Fail'}`;

console.log(message);
```

Use template literals instead of many + operators when building longer strings.

ES6 Modules: export and import

- Modules split a large program into smaller files.
- export makes code available to another module.
- import brings exported code into another file.
- Named exports use matching names.
- default export provides one main value from a module.
- Modules help organize and reuse code.

Two-file example

```
// math.js
export function add(a, b) {
  return a + b;
}

export const tax = 0.15;

// app.js
import { add, tax } from './math.js';

console.log(add(2, 3));
console.log(tax);
```

Putting the Concepts Together

- Use const/let for clear variable management.
- Use an object to represent a student.
- Use an array method to calculate or filter data.
- Use a function with a default parameter.
- Use a template literal to display the result.

Mini practical example

```
const student = {
  name: 'Sara',
  marks: [72, 85, 91]
};

function average(values = []) {
  return values.reduce((a, b) => a + b, 0) /
values.length;
}

const avg = average(student.marks);
const result = avg >= 50 ? 'Pass' : 'Fail';

console.log(`${student.name}: ${avg.toFixed(1)}
- ${result}`);
```